

# Python\_Übung\_1

November 8, 2024

## 1 Datenauswertung mit Python - Übung 1

### 1.1 Aufgabe 1: Verdünnungsreihe

#### 1.1.1 Funktionen: Verdünnung

Berechne aus der gegebenen Konzentration der Stammlösung, der Konzentration der verdünnten Säure und dem Zielvolumen **die Menge and Stammlösung** die benötigt wird. Nutze dazu eine **Funktion** `get_dilution`.

```
[ ]: def get_dilution(c_start, c_target, v_target):  
    """  
    TODO: Berechne das Volumen v_start und das Verdünnungsvolumen v_water, dass  
    ↳ benötigt wird  
    """  
    return v_start, v_water  
  
# Input Variablen  
concentration_available = 12.39 # Konzentration Stammlösung, mol/L  
concentration_target = 0.2 # Zielkonzentration, mol/L  
volume_target = 1.0 # Zielvolumen, L  
"""  
#TODO: Rufe die Funktion get_dilution mit den entsprechenden Argumenten auf  
"""  
get_dilution()  
"""  
#TODO: Füge die entsprechenden Variablen ein  
"""  
print(f"Für VOLUMEN L einer Säure von KONZENTRATION mol/L " \  
      f"müssen VOLUMEN_SÄURE L Säure und VOLUMEN_WASSER L Wasser gemischt, ↳  
      ↳ werden.")
```

#### 1.1.2 For-Loops: Berechnung mehrerer Zielkonzentrationen

Wiederhole die Rechnung aus der vorigen Aufgabe für alle Zielkonzentrationen in der Liste `concentrations` in einem **for-Loop**. Nutze dabei die Funktion `get_dilution` innerhalb des Loops.

```
[ ]: concentration_available = 12.39 # Konzentration Stammlösung, mol/L  
volume_target = 1 # Zielvolumen, L
```

```

concentrations = [0.1, 0.5, 0.75, 1, 5, 10] # # Zielkonzentration, mol/L

# TODO: Loop vervollständigen
for in :
    get_dilution()
    print(f"Für VOLUMEN L einer Säure von KONZENTRATION mol/L " \
          f"müssen VOLUMEN_SÄURE L Säure und VOLUMEN_WASSER L Wasser gemischt_
↪werden.")

```

### 1.1.3 Bedingungen: Schließe zu hohe Konzentrationen aus

Wenn die Zielkonzentration höher ist als die der Stammlösung, ist es nicht möglich diese durch Verdünnung herzustellen. Schließe diese Fälle aus indem du in den Code aus der vorigen Aufgabe eine **if/else** Bedingung einbaust, die prüft ob die Zielkonzentration höher ist als die der Stammlösung.

```

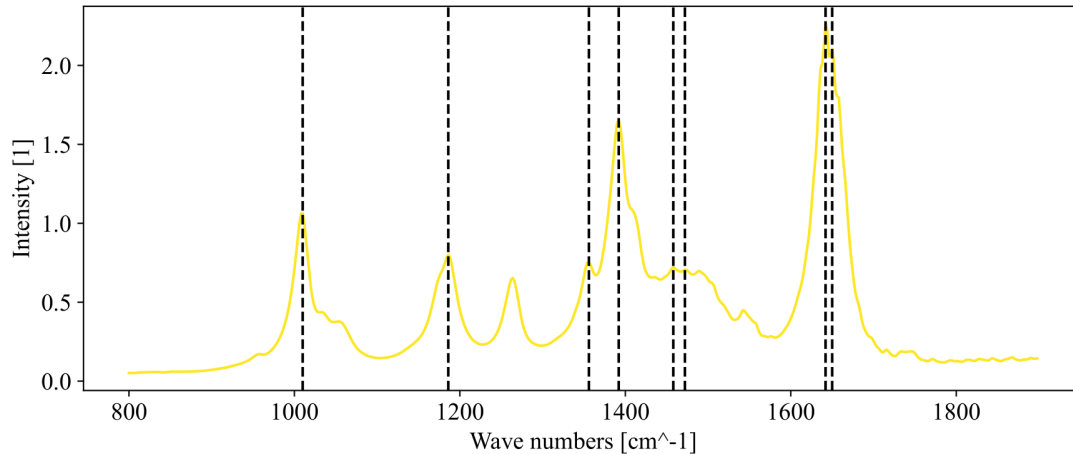
[ ]: concentration_available = 12.39 # mol/L
volume_target = 1 # L
concentrations = [0.1, 0.5, 0.75, 1, 5, 10, 15] # mol/L
"""
TODO: Vervollständige den Loop aus der letzten Aufgabe
"""
for ___ in ___:
    """
    TODO: Prüfe ob die Zielkonzentration größer als die Ausgangskonzentration_
↪ist
    """
    if BEDINGUNG:
        print(f"Die Zielkonzentration von {concentration_target:4.3f} mol/L ist_
↪höher als die vorhandene {concentration_available:4.3f} mol/L!")
    else:
        """
        TODO: Füge die entsprechenden Argumente in die Funktion ein, siehe Teil_
↪1
        """

        get_dilution()
        print(f"Für VOLUMEN L einer Säure von KONZENTRATION mol/L " \
              f"müssen VOLUMEN_SÄURE L Säure und VOLUMEN_WASSER L Wasser_
↪gemischt werden.")

```

## 1.2 Aufgabe 2: Spektrum analysieren

Für die Übung ist ein IR-Spektrum von Dimethylacetamid (DMAc) gegeben.



###

Daten Laden Für das Laden der Daten ist eine Funktion vorgegeben, die die Daten des Spektrums als Listen mit den x- und y-Werten lädt. Nutze die Funktion, um die x- und y- Werte des Spektrums aus der Datei `dmac.csv` zu laden.

```
[ ]: def load_spectrum(filename, separator=","):
    """
    Lädt ein .csv Spektrum als Liste
    filename: String mit Dateiname
    separator: Trenner der x,y Daten in der Datei
    Output:
    x: Liste der Wellenzahlen
    y: Liste der Intensitäten
    """
    x_list = []
    y_list = []
    with open(filename) as file:
        for line in file:
            x, y = line.split(separator)
            x_list.append(float(x))
            y_list.append(float(y))

    return x_list, y_list

filename = "dmac.csv"
"""
TODO
"""
```

### 1.2.1 Finde die Position des größten Peaks

Schreibe eine Funktion die das Absorptionsmaximum und die Wellenzahl des Absorptionsmaximum findet.

### 1.2.2 Hinweise

- Es wird eine **if/else** Bedingung benötigt
- Es kann gleichzeitig über den Index einer Liste und die Elemente geloopt werden:

```
for idx, element in enumerate(some_list):
```

- der Index `max_idx` kann am Ende genutzt werden um die entsprechenden Elemente der Listen aufzurufen

```
[ ]: def peak_maximum(x, y):
    """
    Findet den x- und y- Wert für das Maximum der y-Werte
    x: Liste der Wellenzahlen
    y: Liste der Intensitäten
    Output:
    x_max: Wellenzahl des Absorptionsmaximum
    y_max: Absorptionsmaximum
    """
    max_idx = 0 # Index des größten Elements
    max_y = 0 # Startwert, das Maximum ist größer als 0
    for ____ in ____:
        """
        TODO: Update max_idx und max_y um am Ende den Index des größten
        ↪Elements zu finden
        """
    return x_max, y_max

x_max, y_max = peak_maximum(x, y)

print(f"Das Maximum des größten Peaks liegt bei {x_max:.3f} cm-1")
```

### 1.2.3 Finde die Positionen der anderen markierten Peaks in der Abbildung

Auch wenn das Spektrum mehrere Peakmaxima aufweist, kann die Funktion `peak_maximum` nur das größte im Spektrum finden. Da wir aber wissen **wo** die verschiedenen Maxima auftauchen, können wir den Suchbereich eingrenzen um die gewünschten Peaks zu erhalten. Wir möchten die exakten Positionen von 4 Peaks finden: - Zwischen 900 cm<sup>-1</sup> und 1100 cm<sup>-1</sup> - Zwischen 1100 cm<sup>-1</sup> und 1300 cm<sup>-1</sup> - Zwischen 1300 cm<sup>-1</sup> und 1500 cm<sup>-1</sup> - Zwischen 1600 cm<sup>-1</sup> und 1800 cm<sup>-1</sup>

Die Positionen der Peaks kann bestimmt werden, indem zunächst das Spektrum mit der Funktion `mask_spectrum` auf einen Teilbereich eingegrenzt wird, um danach innerhalb dieses Teilbereichs mit `peak_maximum` das Maximum zu finden.

### 1.2.4 Hinweise:

- List comprehensions können nützlich und effizient sein:

```
some_list = [element for element in Liste if Bedingung ]
```

- Über mehrere Listen kann mit der `zip` Funktion geloopt werden:

```
for el1, el2 in zip(list1, list2):
```

```
[ ]: def mask_spectrum(x, y, x_min, x_max):  
    """  
    Entfernt den Teil des Spektrums der nicht zwischen x_min und x_max liegt  
    x: Liste der Wellenzahlen  
    y: Liste der Intensitäten  
    x_min: Kleinster X-Wert im Ausschnitt des Spektrums  
    x_max: Größter x-Wert im Ausschnitt des Spektrums  
    Output:  
    x_masked: Wellenzahlen im Bereich x_min bis x_max  
    y_masked: Absorption im Bereich x_min bis x_max  
    """  
  
    x_masked = []  
    y_masked = []  
  
    return x_masked, y_masked  
  
peak_x_min = [900, 1100, 1300, 1600]  
peak_x_max = [1100, 1300, 1500, 1800]  
  
# TODO: Bestimme die Peakmaxima in einem For-Loop
```